ROBOT OPERATING SYSTEM-ROS2

1. Introduction to ROS 2

What is ROS 2?

ROS 2, or Robot Operating System 2, is the next-generation version of ROS that builds upon the lessons learned from ROS 1 while introducing significant improvements. It is designed to provide a robust, flexible, and scalable framework for building robotic applications. ROS 2 focuses on addressing the limitations of ROS 1, particularly around real-time capabilities, multi-platform support, and scalability.

ROS 2 aims to provide better support for industrial applications, especially for real-time, high-performance, and safety-critical robotic systems. It uses DDS (Data Distribution Service) for communication, allowing for better scalability, fault tolerance, and flexibility.

Key Features of ROS 2:

- **Real-Time Communication**: Built with real-time constraints in mind, making it suitable for mission-critical robotic applications.
- **Cross-platform Support**: ROS 2 supports multiple operating systems like Linux, Windows, and macOS, unlike ROS 1 which was Linux-centric.
- **Security**: Built-in security features like authentication, encryption, and access control.
- **Multi-Vendor Middleware**: ROS 2 supports DDS middleware, which is flexible and provides better interoperability between different systems and hardware.
- **Masterless Architecture**: Unlike ROS 1, ROS 2 doesn't require a central master node for communication between nodes.

Overview of ROS (Robot Operating System)

The **Robot Operating System (ROS)** is a set of software libraries and tools that help developers build robotic applications. It provides a communication framework for building distributed systems, making it easier to interface with hardware, control robot behaviors, simulate robots, and develop higher-level capabilities.

Key Components of ROS:

- **Nodes**: ROS applications are broken down into nodes. Each node can be a separate process or a thread and is responsible for a specific task, like controlling motors or processing sensor data.
- **Topics**: Nodes can publish messages to topics, which other nodes can subscribe to in order to receive the data. This allows for communication between different parts of the robotic system.
- **Services**: In addition to topics, ROS also supports service-based communication, where one node can send a request to another and wait for a response.
- **Messages**: The data exchanged between nodes is organized into messages, which are defined in .msg files in ROS.
- **Actions**: Actions provide a way to perform long-running tasks with feedback. This is useful for operations like moving a robot to a target location, where the progress can be monitored in real-time.

Differences Between ROS 1 and ROS 2

ROS 2 brings several significant improvements and changes to overcome the limitations of ROS 1:

Feature	ROS 1	ROS 2
Middleware	ROS 1 uses custom communication protocol	ROS 2 uses DDS (Data Distribution Service) for communication
Real-Time	Limited real-time capabilities	Built-in support for real-time operations
Master Node	Central master node required	Masterless architecture (uses DDS for direct peer-to-peer communication)
Cross-Platform	Primarily Linux-based	Cross-platform (Linux, Windows, macOS)
Security	No built-in security features	Built-in security with DDS (encryption, authentication)
Quality of Service (QoS)	Limited QoS features	Enhanced QoS for controlling communication reliability, latency, and bandwidth
Lifecycle Management	Limited lifecycle management	Enhanced node lifecycle management for better control of resources
Support for Multi-Vendor	Limited interoperability	DDS middleware allows for better integration with multiple vendors and hardware

ROS 2 Versions

ROS 2 versions are released periodically with new features, fixes, and improvements. Some key versions are:

- **ROS 2 Dashing Diademata**: The first long-term support (LTS) release in ROS 2, introduced in May 2019.

- **ROS 2 Eloquent Elusor**: Released in November 2019, with several improvements in stability and features.
- **ROS 2 Foxy Fitzroy**: The second LTS release, introduced in June 2020. It brought long-term support and improved stability and performance for industrial applications.
- **ROS 2 Galactic Geochelone**: Released in May 2021, it introduced new features and improvements for ROS 2 in terms of usability and stability.
- **ROS 2 Humble Hawksbill**: The latest LTS release as of 2022, with new improvements in node management, real-time performance, and better cross-platform support.
- **ROS 2 Iron Irwini (Upcoming)**: Scheduled for release in mid-2023, with further improvements in multi-robot support, real-time capabilities, and tooling.

Each release has a support period (usually 1-2 years), with LTS releases being supported for a longer duration (5 years for Foxy, Humble, and future releases).

ROS 2 Architecture

Nodes

In ROS 2, nodes are the fundamental building blocks of a robotic system. A node is a process that performs a computation or action, such as controlling a motor, reading sensor data, or processing images.

Nodes can:

- Communicate with other nodes via topics, services, or actions.
- Be written in either C++ or Python.
- Be created and managed with the ROS 2 command-line tools (ros2 run, ros2 launch).

Nodes can also have **lifecycle management**, meaning that a node can be transitioned between different states (e.g., inactive, active, shutting down). This allows for better control over resources in a robotic system.

Topics, Services, and Actions

- **Topics**: The topic-based communication in ROS 2 allows nodes to send and receive messages asynchronously. A node can **publish** a message to a topic, and other nodes can **subscribe** to that topic to receive the messages. Topics are used for continuous or periodic data exchanges, such as sensor data or status updates.
- **Services**: Services allow synchronous communication between nodes. A service server waits for requests, processes the request, and sends back a response. A service client can make a request and wait for the server's response. Services are ideal for tasks that require a request/response pattern, such as setting a robot's position or controlling a specific robot feature.
- **Actions**: Actions are similar to services, but they are designed for long-running tasks. Actions allow clients to send a request and receive feedback during the operation, along with the final result once completed. Actions are useful for tasks like moving a robot to a destination, where you want to monitor the progress and possibly cancel or modify the action.

ROS 2 Communication Model (DDS)

ROS 2 adopts the **Data Distribution Service (DDS)** as its underlying communication framework. DDS is an open standard for real-time, publish-subscribe communication used in many industries for mission-critical applications. DDS provides several key advantages over the traditional ROS 1 communication model:

- **Peer-to-Peer Communication**: DDS allows direct communication between nodes, eliminating the need for a central master node like in

ROS 1. This increases scalability and reliability, especially in large robotic systems.

- **Quality of Service (QoS)**: DDS offers flexible QoS policies that allow you to configure parameters like reliability, durability, and latency of communication, which is critical for real-time systems.
- **Security**: DDS includes built-in security mechanisms like authentication, encryption, and access control, providing a more secure communication model for robotic applications.
- **Scalability and Fault Tolerance**: DDS handles complex, distributed systems with ease and ensures robustness in large-scale or fault-tolerant systems.

Masterless System

One of the key differences in ROS 2 compared to ROS 1 is the **masterless architecture**. In ROS 1, the system relies on a central **ROS Master** to coordinate communication between nodes. The master is responsible for managing node registration and topic connections.

In contrast, ROS 2 uses DDS, which allows for **peer-to-peer communication** between nodes without the need for a central master. This not only simplifies the architecture but also improves scalability, flexibility, and robustness, as nodes can directly discover and communicate with each other without depending on a single point of failure.

ROS 2 Middleware (DDS, RTPS)

ROS 2's communication is powered by DDS (Data Distribution Service), a middleware standard used for real-time, reliable, and scalable communication. DDS offers several key features:

- **RTPS (Real-Time Publish-Subscribe)**: ROS 2 uses the RTPS protocol, a part of DDS, for real-time communication. RTPS enables

communication between nodes with low-latency and high-reliability, which is essential for real-time applications like robotics.

- **Customizable QoS**: DDS allows for defining Quality of Service (QoS) settings that control message delivery policies, such as reliability, durability, and message size.
- **Flexible Data Exchange**: DDS supports various data exchange models, including point-to-point communication and multicast, providing flexibility for different types of robotic systems.

ROS 2 Workspace Creation

A **ROS 2 workspace** is a directory where ROS 2 packages are organized, built, and managed.

It typically contains:

```
<workspace>/
 ├── src/        # Source folder for ROS 2 packages
 ├── build/      # Build system files (created after build)
 ├── install/    # Installed products (created after build)
 └── log/        # Logs from build process
```

Steps to Create a ROS 2 Workspace

1. Create the Workspace Directory

Create a new workspace directory with a src subfolder:

```
mkdir -p ~/ros2_ws/src
```

- ~/ros2_ws is the name of the workspace (can be renamed as needed, for example my_robot_ws).
- src is the subfolder where all ROS 2 packages will be placed.

2. Navigate to the Workspace

Move into the workspace directory:

```
cd ~/ros2_ws
```

3. Build the Workspace

Use **colcon**, the ROS 2 build tool, to build the workspace:

```
colcon build
```

This command:

- Searches the src/ directory for ROS 2 packages.

- Compiles them.
- Creates the build/, install/, and log/ directories.

4. Source the Workspace

After building, you need to **source the environment** to use the workspace:

```
source install/setup.bash
```

Notes:

- This step must be repeated in each terminal where you want to use the workspace.
- To make sourcing automatic, add the following to your .bashrc:

```
echo "source ~/ros2_ws/install/setup.bash" >> ~/.bashrc
```

ROS 2 Package Creation

In ROS 2, a **package** is the fundamental unit of organization, containing nodes, libraries, configuration files, launch files, and other resources. All ROS 2 development is done inside packages.

Steps to Create a ROS 2 Package

1. Navigate to the src Directory

Go to the src folder of your ROS 2 workspace:

```
cd ~/ros2_ws/src
```

This ensures the package will be placed inside the workspace.

2. Create the Package

Use the ros2 pkg create command.

For a **Python package**:

```
ros2 pkg create --build-type ament_python <package_name>
```

For a **C++ package**:

```
ros2 pkg create --build-type ament_cmake <package_name>
```

Example (Python):

```
ros2 pkg create --build-type ament_python my_robot_pkg
```

Example (C++):

```
ros2 pkg create --build-type ament_cmake my_cpp_robot_pkg
```

Common Options

- --dependencies → List of dependent packages (separated by space)

Example:

```
ros2 pkg create --build-type ament_python my_robot_pkg --dependencies rclpy std_msgs
```

- --license → Specify a license

Example:

```
ros2 pkg create --build-type ament_cmake my_cpp_pkg --license Apache-2.0
```

3. Build the Package

Return to the root of the workspace:

```
cd ~/ros2_ws
```

Then build:

```
colcon build
```

4. Source the Workspace

After building, source the workspace:

```
source install/setup.bash
```

5. Verify the Package

List available packages to check if your new package is detected:

```
ros2 pkg list | grep <package_name>
```

Example:

```
ros2 pkg list | grep my_robot_pkg
```

Generated Package Structure

For a Python package (ament_python), the structure typically looks like:

```
my_robot_pkg/
 ├── package.xml
 ├── resource/
 ├── setup.cfg
 ├── setup.py
 ├── test/
 └── my_robot_pkg/
     └── __init__.py
```

For a C++ package (ament_cmake), it typically looks like:

```
my_cpp_robot_pkg/
 ├── CMakeLists.txt
 ├── package.xml
 └── src/
```

Important Files

- package.xml → Defines package metadata, dependencies, and build info.
- CMakeLists.txt → Build configuration file (for C++ packages).
- setup.py and setup.cfg → Python packaging files (for Python packages).

Example Commands

- **Create Python package with dependencies**:

```
ros2 pkg create --build-type ament_python my_robot_pkg --dependencies rclpy std_msgs
```

- **Create C++ package with dependencies**:

```
ros2 pkg create --build-type ament_cmake my_cpp_robot_pkg --dependencies rclcpp std_msgs
```

Adding Nodes or Code

After package creation:

- Python nodes go inside the <package_name>/ directory.
- C++ source files go inside src/ and are referenced in CMakeLists.txt.

Do not forget to update:

- package.xml with any additional dependencies.
- setup.py (Python) or CMakeLists.txt (C++) when adding executables or libraries.

Troubleshooting

- If package does not appear after colcon build, check:
  - You are inside the correct workspace.
  - You have sourced the install/setup.bash.
  - There are no syntax or configuration errors in package.xml or setup.py/CMakeLists.txt.

If you want, I can also prepare **ready-to-use code templates** or **package skeletons** for common use cases (e.g., a publisher-subscriber package, service-action package). Let me know if you'd like those.

ROS 2 Node Creation

In ROS 2, a **node** is an executable that communicates with other nodes using topics, services, or actions. Nodes are organized inside ROS 2 packages.

Below are the steps for creating nodes in both **Python** and **C++**.

Python Node Creation

1. Prerequisites

- ROS 2 workspace and package already created (with ament_python).
- Dependencies such as rclpy declared in package.xml and setup.py.

2. Create Python Node File

Inside your package directory:

```
cd ~/ros2_ws/src/my_robot_pkg/my_robot_pkg/
```

Create a new Python script, for example:

```
touch simple_node.py
chmod +x simple_node.py
```

3. Example Python Node Code

File: simple_node.py

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class SimpleNode(Node):
    def __init__(self):
        super().__init__('simple_node')
        self.get_logger().info('SimpleNode has been started')

def main(args=None):
    rclpy.init(args=args)
    node = SimpleNode()
```

```
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

4. Register Executable in setup.py.

Edit the setup.py file in your package:

```
entry_points={
    'console_scripts': [
        'simple_node = my_robot_pkg.simple_node:main',
    ],
},
```

This tells ROS 2 how to launch the node.

5. Build the Package

From the workspace root:

```
cd ~/ros2_ws
colcon build
```

6. Source the Workspace

```
source install/setup.bash
```

7. Run the Node

```
ros2 run my_robot_pkg simple_node
```

C++ Node Creation

1. Prerequisites

- ROS 2 workspace and package already created (with ament_cmake).
- Dependencies such as rclcpp declared in package.xml and CMakeLists.txt.

2. Create C++ Source File

Inside your package:

```
cd ~/ros2_ws/src/my_cpp_robot_pkg/src/
touch simple_node.cpp
```

3. Example C++ Node Code

File: simple_node.cpp

```
#include "rclcpp/rclcpp.hpp"
class SimpleNode : public rclcpp::Node
{
public:
    SimpleNode() : Node("simple_node")
    {
        RCLCPP_INFO(this->get_logger(), "SimpleNode has been started");
    }
};
int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<SimpleNode>());
    rclcpp::shutdown();
    return 0;
}
```

4. Update CMakeLists.txt

- Add the executable and link libraries:

```
add_executable(simple_node src/simple_node.cpp)
ament_target_dependencies(simple_node rclcpp)
```

```
install(TARGETS
  simple_node
  DESTINATION lib/${PROJECT_NAME})
```

5. Build the Package

From the workspace root:

```
cd ~/ros2_ws
colcon build
```

6. Source the Workspace

```
source install/setup.bash
```

7. Run the Node

```
ros2 run my_cpp_robot_pkg simple_node
```

ROS 2 Simple Node

We will create:

- Package: my_robot_pkg
- Python node: simple_node.py

1. Navigate to Workspace

```
cd ~/ros2_ws/src
```

2. Create the Package

```
ros2 pkg create --build-type ament_python my_robot_pkg --dependencies rclpy
```

This creates the package folder my_robot_pkg/ with necessary files.

3. Write the Node Script

Navigate to the package's internal directory:

```
cd ~/ros2_ws/src/my_robot_pkg/my_robot_pkg
```

Create the Python node file:

```
touch simple_node.py
chmod +x simple_node.py
```

4. Add Node Code

Edit simple_node.py and add:

```
#!/usr/bin/env python3

import rclpy
from rclpy.node import Node

class SimpleNode(Node):
    def __init__(self):
        super().__init__('simple_node')
        self.get_logger().info('SimpleNode has been started')

def main(args=None):
    rclpy.init(args=args)
    node = SimpleNode()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

5. Update setup.py

Go to the package root:

```
cd ~/ros2_ws/src/my_robot_pkg
```

Edit setup.py and make sure you have this under entry_points:

```
entry_points={
```

```
    'console_scripts': [
        'node_simple = my_robot_pkg.simple_node:main',
    ],
},
```

This tells ROS 2 how to run the script as a command.

6. Build the Workspace

From the workspace root:

```
cd ~/ros2_ws
colcon build
```

7. Source the Setup

```
source install/setup.bash
```

8. Run the Node

```
ros2 run my_robot_pkg node_simple
```

Expected terminal output:

[INFO] [<timestamp>] [simple_node]: SimpleNode has been started

Folder Structure After Setup

```
ros2_ws/
 ├── src/
 │   └── my_robot_pkg/
 │       ├── my_robot_pkg/
 │       │   └── simple_node.py
 │       ├── package.xml
 │       ├── setup.py
 │       ├── setup.cfg
 │       └── resource/
 ├── build/
```

```
├── install/
└── log/
```

<u>Continuous ROS 2 Python Node</u>

Navigate to Package Directory

```
cd ~/ros2_ws/src/my_robot_pkg/my_robot_pkg
touch continuous_node.py
chmod +x continuous_node.py
```

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class ContinuousNode(Node):
    def __init__(self):
        super().__init__('continuous_node')
        # Set timer: 1.0 second (1000 ms)
        self.timer = self.create_timer(1.0, self.timer_callback)
        self.get_logger().info('ContinuousNode has been started')

    def timer_callback(self):
        self.get_logger().info('ContinuousNode is running...')

def main(args=None):
    rclpy.init(args=args)
    node = ContinuousNode()
    rclpy.spin(node)
    node.destroy_node()
```

```
    rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Update setup.py

Edit setup.py and make sure you have this under entry_points:

```
entry_points={
    'console_scripts': [
        'simple_node = my_robot_pkg.simple_node:main',
        'continuous_node = my_robot_pkg.continuous_node:main',
    ],
},
```

Run the Continuous Node

ros2 run my_robot_pkg continuous_node

ROS COMMUNICATION

ROS 2 Publisher and Subscriber – Step-by-Step (Python)

Step 1: Create a ROS 2 Python Package

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python simple_pubsub --dependencies rclpy
std_msgs
```

This creates a folder simple_pubsub with necessary files.

Step 2: Write the Publisher Node

Create the file simple_publisher.py inside the package:

```
cd ~/ros2_ws/src/simple_pubsub/simple_pubsub
gedit simple_publisher.py
```

simple_publisher.py:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')
        self.publisher_ = self.create_publisher(String, 'topic', 10)
        timer_period = 1  # seconds
        self.timer = self.create_timer(timer_period, self.publish_message)
        self.count = 0

    def publish_message(self):
        msg = String()
        msg.data = f'Hello ROS 2! Count: {self.count}'
        self.publisher_.publish(msg)
```

```
        self.get_logger().info(f'Published: "{msg.data}"')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    node = SimplePublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

Step 3: Write the Subscriber Node

Create the file simple_subscriber.py:

```
gedit simple_subscriber.py
```

simple_subscriber.py:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')
        self.subscription = self.create_subscription(
            String,
            'topic',
            self.listener_callback,
            10
        )

    def listener_callback(self, msg):
        self.get_logger().info(f'Received: "{msg.data}"')
```

```
def main(args=None):
    rclpy.init(args=args)
    node = SimpleSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

Step 4: Update setup.py

Open setup.py and add both nodes under entry_points:

```
entry_points={
    'console_scripts': [
        'simple_publisher = simple_pubsub.simple_publisher:main',
        'simple_subscriber = simple_pubsub.simple_subscriber:main',
    ],
},
```

Step 5: Make Python Files Executable

```
chmod +x simple_publisher.py
chmod +x simple_subscriber.py
```

Step 6: Build the Package

Go to your workspace root and build:

```
cd ~/ros2_ws
colcon build
```

Step 7: Source the Workspace

```
source install/setup.bash
```

Step 8: Run the Nodes

In **two terminals**:

Terminal 1 – Run Publisher

```
ros2 run simple_pubsub simple_publisher
```

Terminal 2 – Run Subscriber

```
ros2 run simple_pubsub simple_subscriber
```

You will see:

[INFO] [publisher]: Published: "Hello ROS 2! Count: 0"

[INFO] [subscriber]: Received: "Hello ROS 2! Count: 0"

...